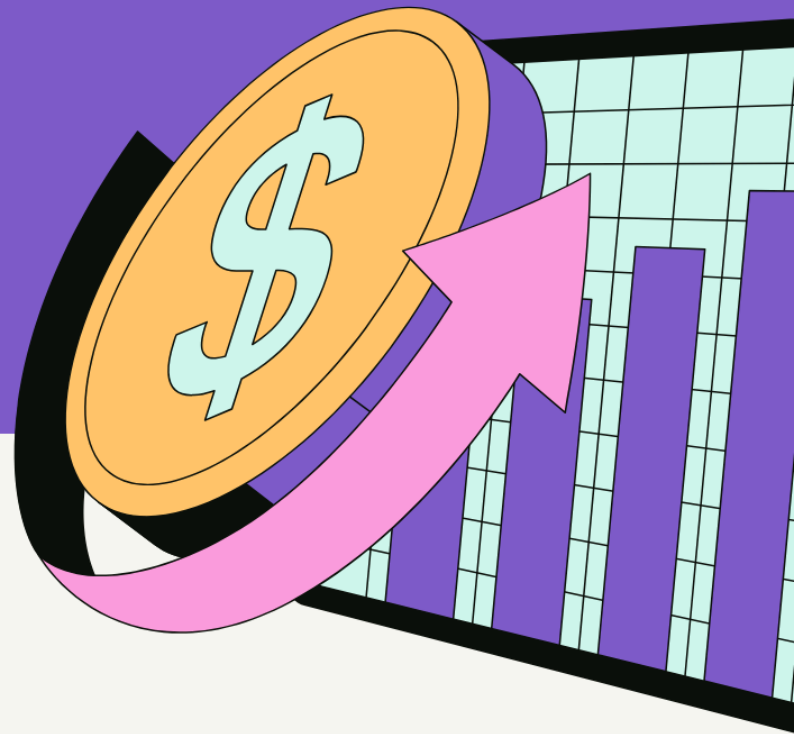


TWEETS-BASED MARKET SENTIMENT ANALYSIS

2025/2026



Group 15

Beatris Daicu, 20221854

Diogo Carvalho, 20221935

Ricardo Pereira, 20250343

Yehor Malakhov, 20221691

Contents

1. Introductions	3
2. Data Exploration	4
3. Data Preprocessing	5
3.1. Emojis	5
3.2. Hashtag Splitting	5
3.3. Lowercasing	5
3.4. URLs, Mentions, and CashTags	5
3.5. Punctuation Cleaning	6
3.6. Tokenization	6
3.7. Stop Words Removal	6
3.8. Lemmatization	6
3.9. Translation	6
4. Feature Engineering	7
4.1. Bag of Words/TF-IDF	7
4.2. Word2Vec/Sentence Embeddings	7
4.3. Transformer Encoder	7
5. Classification Models and Results	8
5.1. Traditional ML Models	8
5.2. Fine-Tuned Transformer Classifiers	8
6. Annexes	9

1. Introductions

This project aimed to develop a Natural Language Processing model that predicts market sentiment from tweets, classifying each as Bearish, Bullish, or Neutral. Financial tweets are typically short, noisy, and context-dependent, presenting a challenging text mining task. Despite these difficulties, such tweets can provide valuable insights into how market news, companies, and economic events are perceived.

To solve this problem, we tested different text preprocessing strategies, feature engineering approaches, and classification models. The project explored traditional machine learning models using Bag-of-Words and Word2Vec representations, as well as transformer-based approaches. Our primary goal was to achieve good predictive performance while also understanding how different text representations influence financial sentiment classification.

2. Data Exploration

We began by analysing the dataset to understand its structure, class distribution, and key textual features. The training set consisted of 9,543 labelled tweets, while the test set included 299 unlabelled tweets. Each training tweet was assigned one of three sentiment labels: 0 (Bearish), 1 (Bullish), or 2 (Neutral).

Before training the models, we split the labelled dataset into training and validation sets using an 80/20 ratio, stratified by sentiment label to maintain class proportions. This resulted in 7,634 tweets for training and 1,909 for validation.

Figure 1 illustrates the sentiment class distribution in the training data. The Neutral class is clearly the most frequent, while the Bearish and Bullish classes are less represented. This class imbalance is relevant for the modelling stage, as classifiers can become biased toward the majority class. Therefore, metrics like macro precision, macro recall, and macro F1-score were prioritised for evaluation.

The raw text showed typical characteristics of tweet data, as many tweets contained URLs, stock mentions, user mentions, hashtags, punctuation, numbers, and financial symbols (Figure 2). There were also many stopwords, which are common words that usually carry limited meaning for classification. The initial word frequency analysis confirmed that the most frequent terms in the raw corpus included several generic words rather than only sentiment-related or financial terms.

After applying the preprocessing pipeline, the vocabulary became more focused. When comparing the raw and cleaned word frequency charts, we see that the cleaning process reduced irrelevant terms while still keeping important financial vocabulary. This is also reflected in the cleaned word cloud, where the most visible terms are more closely related to companies, stocks, market movements, and investor sentiment, like “market”, “stock”, “price”, and “growth”.

Finally, the UMAP visualisations were used to explore how the different feature representations separated the three classes. The plots show that Bearish, Bullish, and Neutral tweets are not perfectly separated, which indicates that this is a challenging classification task. This difficulty is expected because financial tweets can be short, informal, and sometimes ambiguous.

3. Data Preprocessing

Based on what we found during exploration, we built a preprocessing pipeline to clean and normalize the tweet text before feeding it into any model. The pipeline is implemented in `preprocess.py` as a set of functions that each take and return a `pd.Series`.

3.1. Emojis

Rather than simply removing emojis, we convert them to their text descriptions using the emoji library. This is because emojis often carry real sentiment information; for example,  clearly signals a bearish attitude, and  suggests enthusiasm. Throwing that away would mean losing a useful signal.

The `demojize` function replaces each emoji with its name surrounded by spaces (e.g., “” becomes “rocket”). Flag emojis are handled too, as they are made up of Unicode regional indicator letters, so without demojization, they would look like random alphabetic characters.

This step is applied first, before lowercasing, so that the capitalisation in emoji descriptions is available to the hashtag splitter in the next step.

3.2. Hashtag Splitting

Hashtags like `#StockMarketCrash` contain meaningful words but would be treated as a single unknown token without splitting. We split them using a regex that detects word boundaries within the hashtag based on three rules:

Lowercase to uppercase transitions: `#StockMarket` -> “Stock Market”

Uppercase to CamelCase transitions (for acronyms): `#USAUpdates` -> “USA Updates”

Letter to digit transitions: `#SP500` -> “SP 500”

Fully uppercase hashtags like `#ILNYC` are left as a single token, since splitting them letter by letter (I, L, N, Y, C) would produce meaningless noise. This step is done before lowercasing because the splitting logic depends on case.

3.3. Lowercasing

After the steps that depend on capitalisation (emoji demojization and hashtag splitting), we convert everything to lowercase. This prevents the model from treating “Bullish” and “bullish” as different words, which reduces vocabulary size without losing any real information.

3.4. URLs, Mentions, and CashTags

Three types of tokens appear frequently in financial tweets but don’t carry direct sentiment information in their surface form:

- URLs are replaced with the token `url`. Links to news articles don’t convey the sentiment of the tweet and keeping them would clutter the vocabulary with thousands of unique strings.
- User mentions (`@handle`) are replaced with `ref`. The specific account being mentioned is usually not predictive of sentiment.

- Cashtags (\$TICKER) are replaced with stock. While a particular ticker might correlate with sentiment in the training set, we don't want the model to overfit specific tickers that may not appear in the test set.

3.5. Punctuation Cleaning

We expanded contractions and removed/normalized punctuation. Particular attention was given to financial symbols, such as the percentage symbol, which was converted to the word “percent” to preserve the significance of percentage movements in financial contexts. We also changed every type of dash to a single type. We removed the dashes that have no meaning, like dashes between words, but if a dash came before a number, we changed it to the minus character and if a dash exists between numbers, we replaced it with “to”.

3.6. Tokenization

We tokenize the cleaned text using `nlk.word_tokenize`, which splits text into individual word tokens. We chose this over a simple whitespace split because it handles edge cases more reliably. For example, it correctly separates punctuation that might have survived earlier steps.

3.7. Stop Words Removal

We removed stopwords (from NLTK). However, we did not remove all default English stopwords without adjustment. Some words, such as “up”, “down”, “above”, “below”, “more”, “less”, “not”, and “no” were kept because they can be important for market sentiment. For example, “stock up”, “shares down”, or “not profitable” can change the meaning of a tweet.

3.8. Lemmatization

We lemmatize the tokens using the NLTK WordNetLemmatizer. Before lemmatizing, each token is POS-tagged so the lemmatize knows whether to treat it as a noun, verb, adjective, or adverb. This matters because without POS information, a word like “running” would be left unchanged (treated as a noun), whereas with the correct verb tag, it is correctly reduced to “run”.

We chose lemmatization over stemming because it produces real dictionary words. This is important both for readability in word clouds and frequency analyses, and because our downstream word embedding and transformer models were trained on natural text, so they work better with properly formed words than with stemmed fragments like “run” vs “running” vs “runnng”.

3.9. Translation

We noticed that a portion of the tweets is not written in English. To handle this, we used the Lingua library to detect the language of each tweet, and then translated non-English tweets to English using the Google Translate API (via deep-translator).

4. Feature Engineering

After preprocessing the tweets, the next step was to transform the text into numerical representations that could be used by the models. Since machine learning algorithms cannot work directly with raw text, feature engineering was necessary to convert each tweet into a vector while keeping as much useful information as possible.

In this project, we tested three different approaches: Bag-of-Words with TF-IDF, Word2Vec, and Transformer-based embeddings.

4.1. Bag of Words/TF-IDF

The first approach was Bag-of-Words using TF-IDF, where each tweet is represented based on the words that appear in it, while giving more importance to words that are relevant in a specific tweet but not too common across the corpus.

This representation was chosen as a baseline due to its simplicity, rapid training capabilities, and ease of interpretation. In the context of financial tweets, these qualities are especially advantageous, as certain words or expressions are often directly linked to sentiment.

To minimise noise, very rare words were excluded by applying a minimum document frequency threshold. This approach prevented the creation of features from tokens that appeared only a handful of times and were unlikely to generalise to new tweets. Simultaneously, the representation retained the most frequent and relevant terms in the corpus.

However, BoW also has important limitations, as it does not consider the order of words, and it treats each token mostly independently. This means that it may miss some context, especially when the sentiment depends on how words are combined in the sentence.

4.2. Word2Vec/Sentence Embeddings

The second representation evaluated was Word2Vec. Unlike the BoW approach, Word2Vec generates dense vector embeddings, assigning similar vectors to words that frequently occur in comparable contexts.

To make this possible, we used gensim to implement the Word2Vec and trained it on the tokenized dataset using the CBOW (Continuous Bag of Words) architecture.

A vector size of 100 dimensions, a context window of 5 words and minimum word frequency of 1 were established during training. This method enables the capture of semantic relationships between words, moving beyond simple frequency counts.

4.3. Transformer Encoder

The third representation was based on transformer encoders. We used the bert-base-uncased model from Hugging Face's transformers library. As the clean data is translated to English, we don't need a multilingual model. The transformer returns a 768-dimensional vector for each tweet. This representation captures complex contextual relationships between words, as the transformer architecture is designed to understand the meaning of words in relation to the entire sentence, which is particularly beneficial for sentiment analysis as words can have different meanings depending on their context.

5. Classification Models and Results

5.1. Traditional ML Models

On each of the encodings, we applied two models, namely K-Nearest Neighbours and Random Forest Classifiers. For each of the Classifier-Encoding Pairings, we applied a random search with 30 iterations to optimize hyperparameters. You can see the hyperparameters used in Table 1 and 2.

In all three cases Random Forest Classifier was proven to nearly always predict the majority class, as can be seen in Figure 6, 7 and 8.

Similarly, KNN struggled in BoW and Word2Vec encodings; however, using the Transformer encoder, KNN managed to get an F1 score of 0.59 and a recall score of 0.6, and predict all classes equally well. You can compare it to other scores in Table 3.

Nonetheless, as these scores were not satisfactory, we moved on to classifying with Transformers.

5.2. Fine-Tuned Transformer Classifiers

For Transformer Classifiers, we used BERT and DistilBERT models, specifically the bert-base-multilingual-cased and distilbert-base-multilingual-cased models provided by Hugging Face. As mentioned in the Data Exploration (p. 4) section, our data is imbalanced, to address this, we used class weights in the loss function during training. AdamW optimiser was used for training the models, and we trained for 5 epochs with a learning rate of 2×10^{-5} . As opposed to the previous models, for the transformer classifiers, we did not use the preprocessing and feature engineering steps described in the previous sections. Instead, we relied on the tokenization and encoding provided by the models' respective tokenizers. Our reasoning for this decision was that the transformer models would perform better if their tokenizers had access to the raw text. However, as we did not compare the performance of the transformer classifiers with and without preprocessing, we cannot be certain that this was the best choice.

As seen in Table 3, the BERT model achieved ever so slightly better performance than the DistilBERT model. Both models outperformed the traditional machine learning models, as such, for the final submission, we chose to use the BERT model for classification.

6. Annexes

HYPERPARAMETER	VALUES
N-Neighbours	$\{1, 3, 5, \dots, 15\}$
Weights	Uniform, Distance
p	1, 2, 3

Table 1: KNN Hyperparameter Grid

HYPERPARAMETER	VALUES
N-Estimators	$\{10, 20, 30, \dots, 250\}$
Criterion	Gini, Entropy
Max Depth	$\{1, 2, 3, \dots, 10\}$
Min Samples Split	$2^p, p \in \{1, 2, 3, \dots, 10\}$
Min Samples Leaf	$2^p, p \in \{0, 1, 2, \dots, 10\}$
Max Features	Sqrt, Log2, 1

Table 2: Random Forest Hyperparameter Grid

MODEL	ENCODING	ACCURACY	MACRO RECALL	MACRO F1-SCORE
KNN	BoW	0.75	0.61	0.63
Random Forest	BoW	0.67	0.36	0.32
KNN	Word2Vec	0.66	0.47	0.47
Random Forest	Word2Vec	0.69	0.44	0.43
KNN	Transformer Encoder	0.69	0.60	0.59
Random Forest	Transformer Encoder	0.67	0.38	0.36
BERT (Fine-Tuned)	-	0.84	0.82	0.79
DistilBERT (Fine-Tuned)	-	0.84	0.79	0.78

Table 3: Performance Metrics for All Models and Encodings

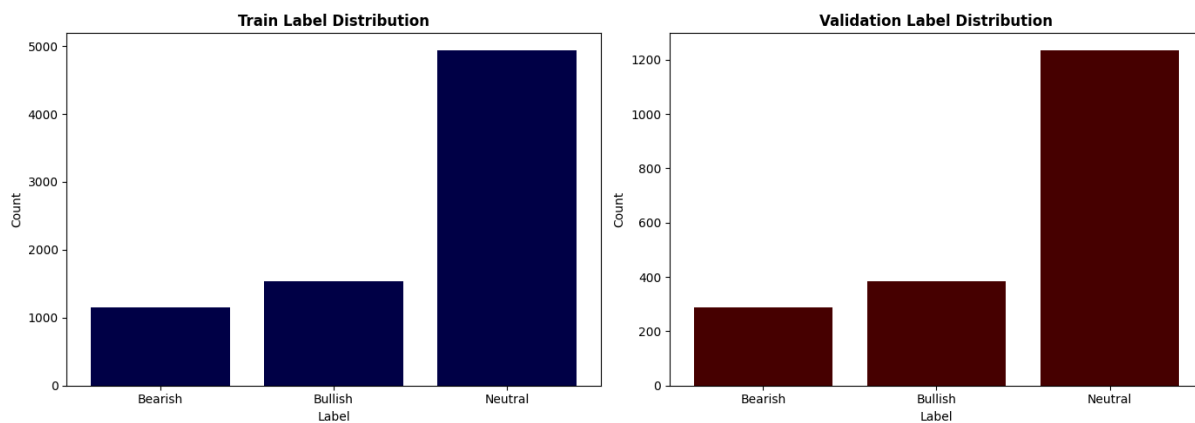


Figure 1: Train and Validation Label Distribution

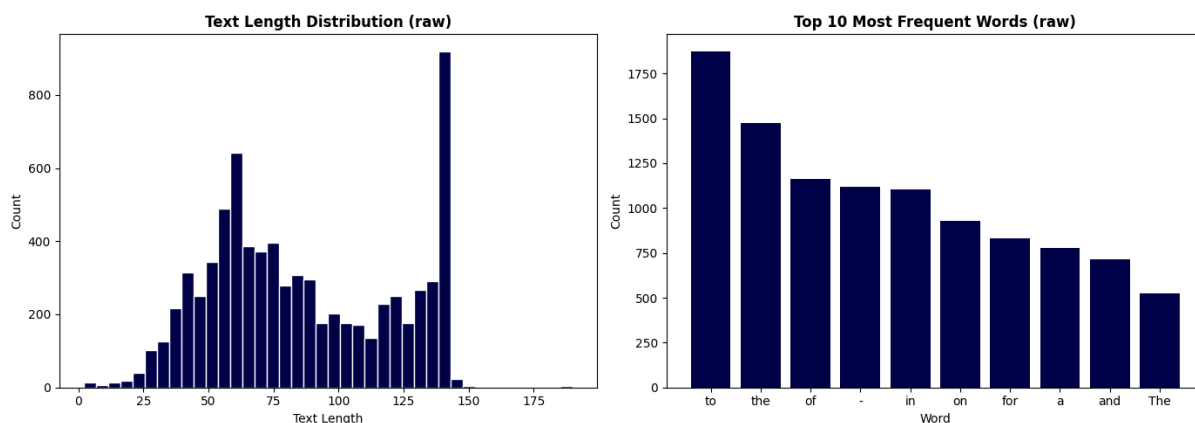


Figure 2: Raw Text Length Distribution and Top 10 Raw Words

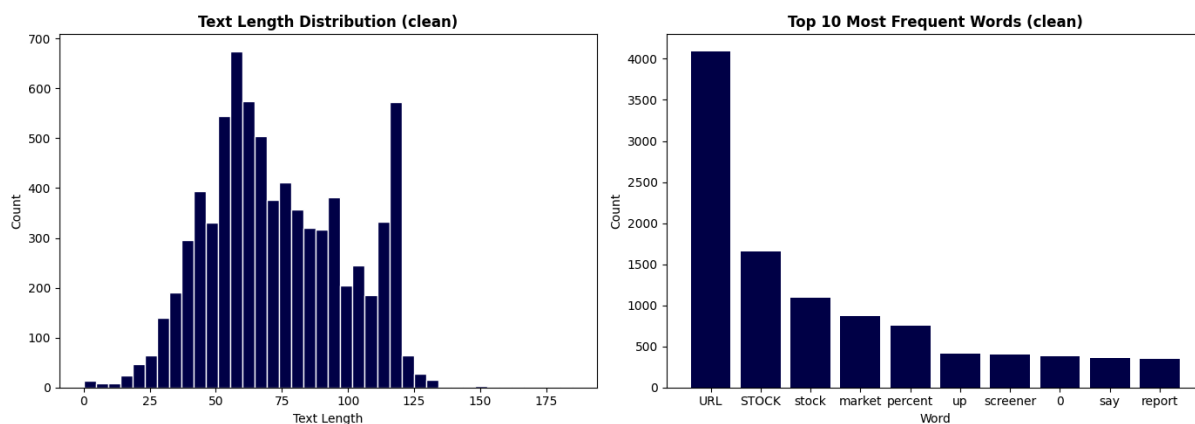


Figure 3: Clean Text Length Distribution and Top 10 Clean Words

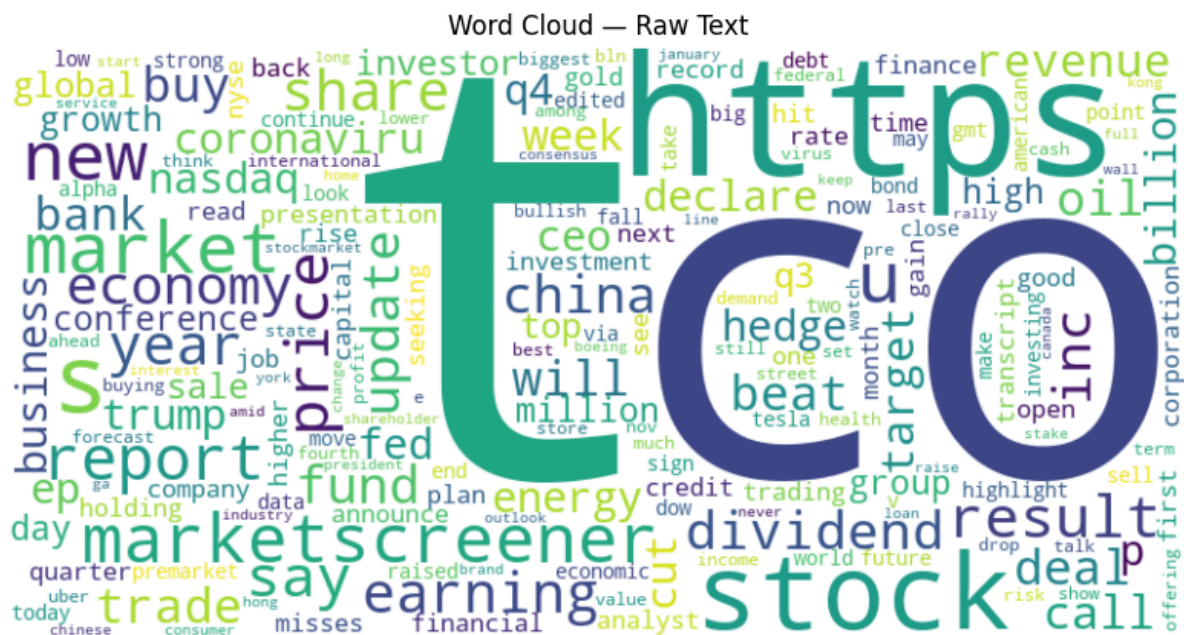


Figure 4: Raw Word Cloud

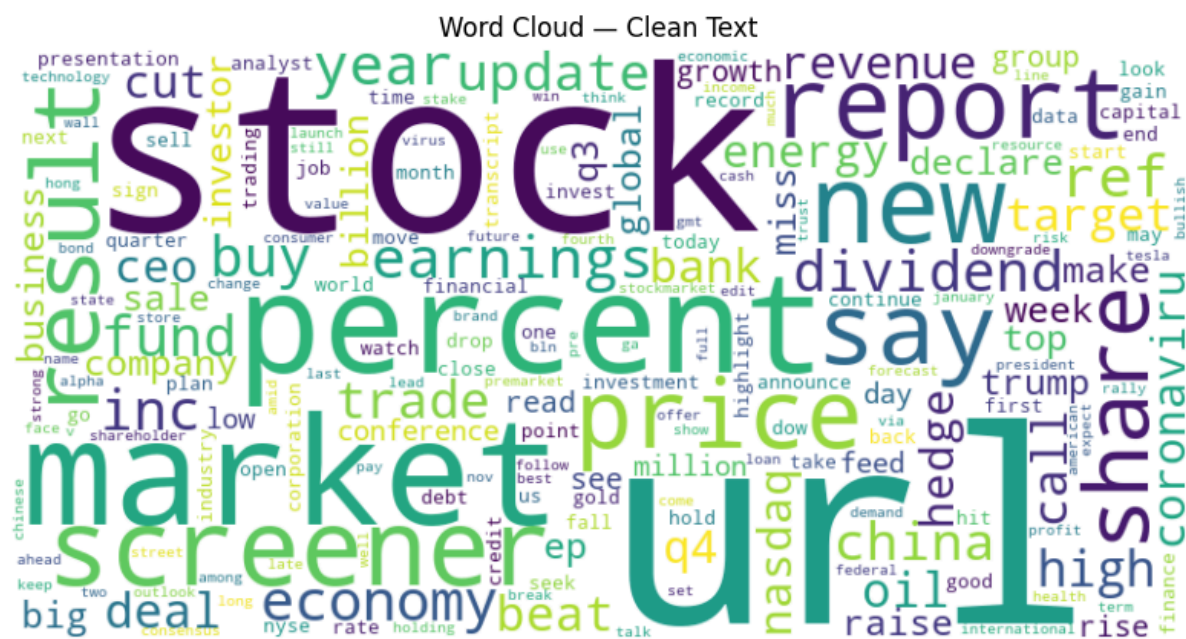


Figure 5: Clean Word Cloud

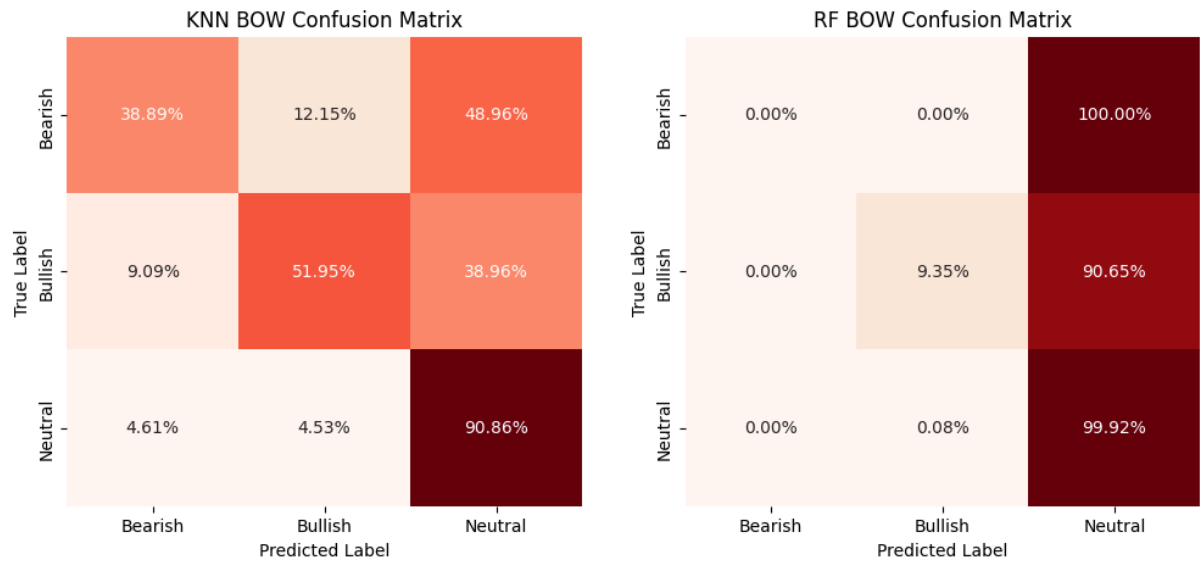


Figure 6: BoW KNN vs RF Confusion Matrices

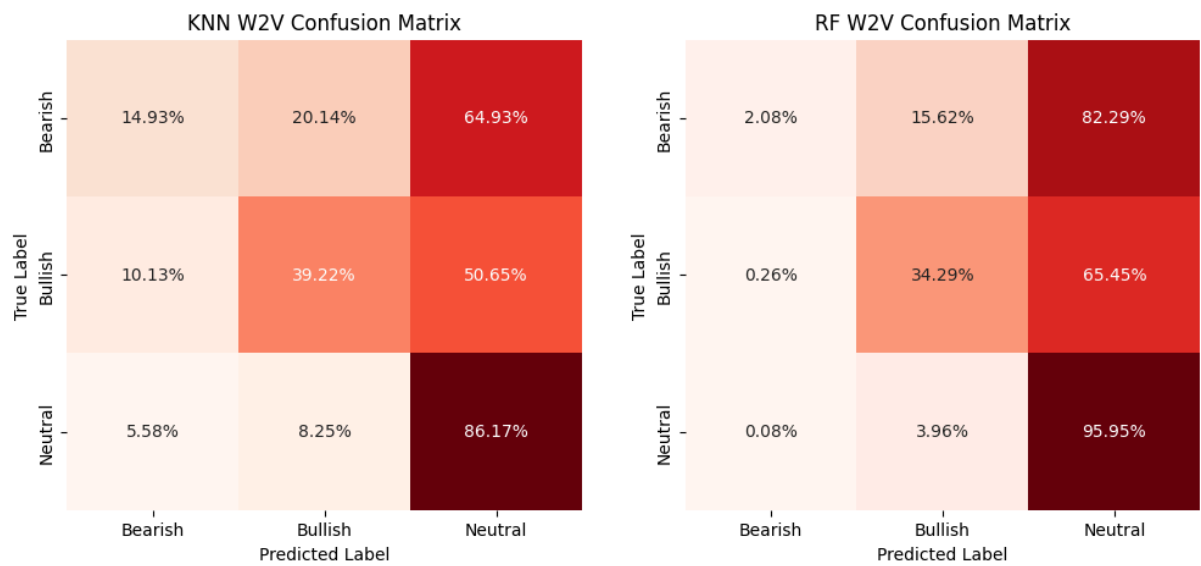


Figure 7: Word2Vec KNN vs RF Confusion Matrices

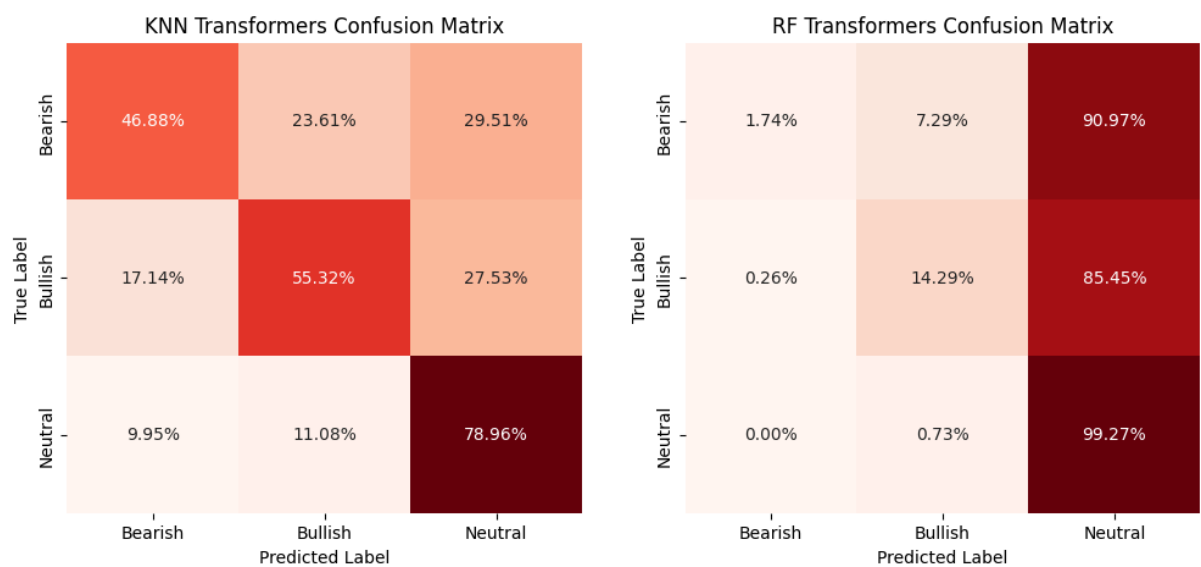


Figure 8: Transformer Encoder KNN vs RF Confusion Matrices

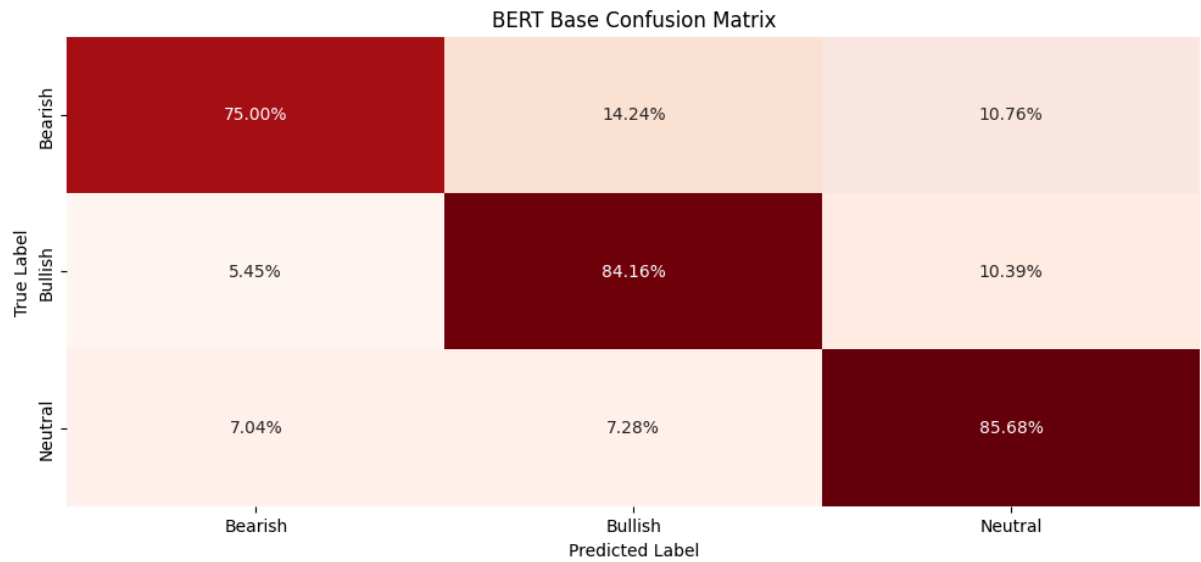


Figure 9: BERT Confusion Matrix

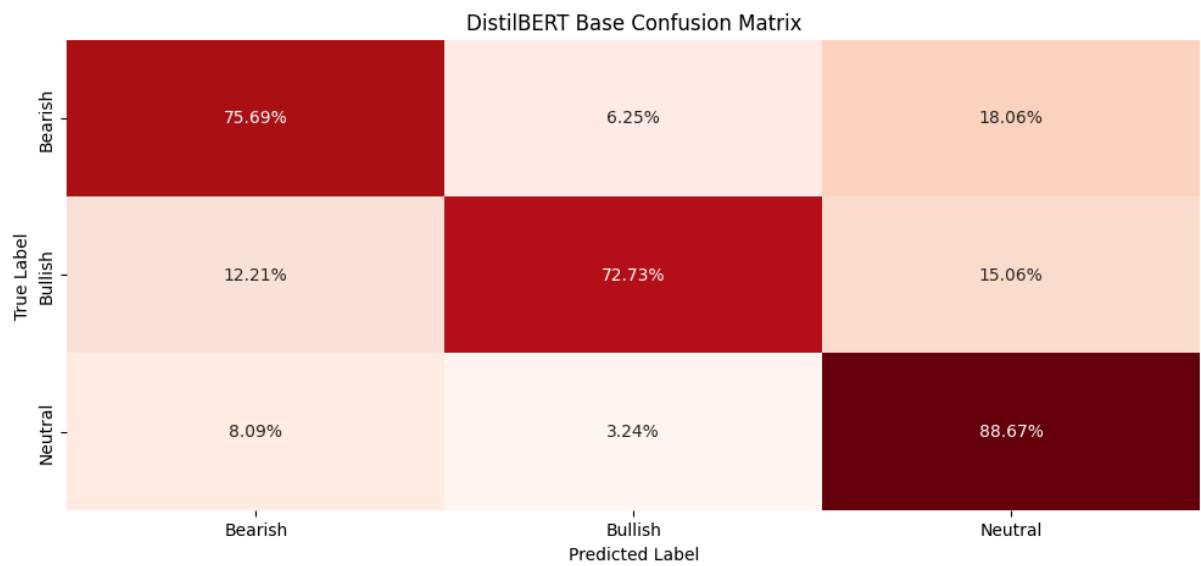


Figure 10: DistilBERT Confusion Matrix